



BACHELOR'S THESIS TEMPLATE - TITLE OF PROJECT

Bachelor's Thesis

Arthur Ernst Henrik Reuss, s5582253, a.e.h.reuss@student.rug.nl,
Supervisors: Gido van de Ven

Abstract: Replay-based continual learning is usually judged by the accuracy it reaches *after* each task. Continual evaluation has exposed a transient failure that this end-of-task view hides: the *stability gap*, a sharp drop in past-task accuracy at the start of every new task, before the model recovers. The gap persists even when the optimiser has access to the exact joint loss over all tasks, so it is not a defect of the objective but of the optimisation trajectory. We argue that, in its dominant components, the gap is a symptom of a single structural feature of online continual learning: at each task boundary the loss the optimiser follows changes discontinuously, from the replay loss alone to the sum of replay and current-task losses. We decompose the gap into three additive contributors — a first-step magnitude asymmetry, a finite-buffer estimator variance, and a trajectory effect — and identify the discontinuity as the source of the first and third. To remove it, we introduce the λ -*curriculum*, a homotopy that ramps the current-task loss weight from 0 to 1 over the first steps of each task, so the optimum moves along a continuous path rather than teleporting across the landscape. An envelope-theorem argument shows the replay loss is monotone non-decreasing along this path, bounding the trajectory contribution by $O(1/N)$ in the ramp length N . We add two refinements, a $\lambda_{\min}$ floor that restores early-task plasticity and momentum that suppresses the residual stochastic oscillation, and evaluate the method on rotated MNIST against vanilla and gradient-balanced Experience Replay.

1 Introduction

Deep neural networks learn rich representations from static, independent, and identically distributed data, but a vulnerability appears as soon as training becomes sequential: the network overwrites previously acquired knowledge to accommodate new data. This phenomenon, catastrophic forgetting [17, 20], is a major hurdle in machine learning. While retraining from scratch solves the issue, it is simply too expensive for systems that need to adapt dynamically. Continual learning (CL) studies how to learn from a sequence of tasks without that cost, and its core difficulty is the stability–plasticity dilemma: the model must stay plastic enough to learn new distributions while staying stable enough to preserve old ones [18].

Several families of techniques address forgetting: replay, parameter and functional regularisation, optimisation-based methods, context-dependent processing, and template-based classification [22]. These methods reduce long-term forgetting, but they have historically been evaluated only at task boundaries or at coarse intervals, which

hides what happens within a task. Evaluating the network continuously exposes a failure these protocols miss. Lange et al. [15] did exactly this and identified the *Stability Gap*, a transient but severe collapse in past-task accuracy that occurs immediately upon the introduction of a new task. Even a well-tuned Experience Replay model that finishes a task with high accuracy on earlier tasks passes through a deep minimum during the first hundred steps of the new task.

This matters for two reasons. Because deployed continual-learning systems are queried throughout training, a model caught mid-transition will commit errors that its final checkpoint never would. Furthermore, this temporary forgetting presents an interesting puzzle: the model clearly retains the knowledge needed to recover, meaning a specific mechanism must disrupt and later restore it. Forcing the model to relearn what it already knows wastes valuable compute, making it essential to understand this underlying mechanism so we can close the gap efficiently.

The discovery of the gap raised an immediate question: could it be removed by better approxi-

imating the joint loss over all tasks? Hess et al. [10] showed that the gap persists even under incremental joint training, where the model has access to all data from all tasks at every step. A perfect objective still produces transient forgetting, so the problem lies not in *what* is optimised but in *how* the optimisation proceeds.

To see why the trajectory itself produces the gap, consider the geometry of sequential optimisation. Figure 1.1 visualises a transition from a learned task (Task A) to a new task (Task B), a conceptual two-dimensional reduction of a high-dimensional surface that nonetheless captures the dynamics at play. Our depiction of the transition geometry borrows the loss-landscape lens of Kao et al. [12], who visualise sequential optimisation in this way to motivate curvature-aware updates; we adopt their geometric framing while drawing the opposite conclusion about how to respond to it (Section 3). The network starts at the converged minimum A of Task A and must reach the joint minimum $A \cap B$. Running stochastic gradient descent (SGD) on the joint loss from A follows the green curve: pulled by the steepest gradients of the joint landscape, the trajectory bows outward, leaving Task A’s basin and crossing a region of higher Task-A error before converging at $A \cap B$. The dip on Task A along this arc is the stability gap.

This trajectory is the consequence of a discontinuity at the task boundary. While Task A is learned the optimiser descends the replay loss alone, the red basin in Figure 1.1. The moment Task B begins, the objective switches in a single step to the sum of the replay and current-task losses, the purple joint basin. Its minimum at $A \cap B$ replaces the red one from one step to the next. The parameters do not move during this switch, yet the minimum they are chasing jumps from θ_A^* to θ_{joint}^* . We call this *landscape teleportation*: from the optimiser’s point of view the surface it was descending disappears and is replaced by another whose minimum lies elsewhere.

The teleportation does more than redirect the optimiser along a curved path; it also makes the very first step after the switch skewed. At θ_A^* the replay gradient is near zero, because the optimiser has converged on Task A, while the current-task gradient is large, because Task B is still unlearned. The summed update is therefore dominated by the new-task gradient, and SGD takes a long step out

of Task A’s basin before the still-negligible replay term can pull it back. This magnitude imbalance compounds the trajectory effect: not only does the descent path bow away from Task A’s minimum, the first step along it is the largest and the least opposed, which is why the dip is steepest in the first updates after a task switch [15]. A finite replay buffer adds a third, smaller error: it estimates the past-task gradient from a handful of stored samples rather than from all past data, so even the replay signal it supplies is noisy. Trajectory effect, magnitude asymmetry, and estimator variance are the three contributors to the gap; Section 3 makes their decomposition precise and the experiments measure each in isolation.

Existing methods take this discontinuity as given. They accept that the objective jumps at the task boundary and instead make the optimiser robust to the jump, constraining each SGD update with information about the past tasks, their gradients or local curvature, so that SGD avoids directions in which the replay loss rises sharply [3, 12, 13, 16]. We group these approaches under the term *path-finding*: while effective, they navigate the broken landscape rather than fixing it.

This thesis takes the opposite route. Rather than steer the optimiser carefully across a discontinuity, we propose removing the discontinuity itself, an approach we call *landscape-shaping*. We instantiate it as the λ -**curriculum**: instead of switching the objective from the replay loss to the joint loss in a single step, the curriculum ramps the current-task loss weight from 0 to 1 over the first N steps of each transition,

$$\begin{aligned} L_\lambda(t) &= L_{\text{replay}} + \lambda(t) L_{\text{current}}, \\ \lambda(t) &= \text{clip}\left(\frac{t}{N}, 0, 1\right), \end{aligned} \quad (1.1)$$

so the optimum $\Theta^*(\lambda)$ traces a continuous curve from θ_A^* (at $\lambda = 0$) to θ_{joint}^* (at $\lambda = 1$), and the model follows this moving valley instead of being teleported across the discontinuity (fourth panel of Figure 1.1). That such a continuous low-loss route from θ_A^* to θ_{joint}^* exists at all is what work on linear mode connectivity [5, 7] establishes. These studies show that distinct optima are often connected by simple paths of near-constant loss. We visualize this path as the dashed corridor in Figure 1.1. The curriculum’s role is to keep the optimiser on this specific route and prevent it from bowing away.

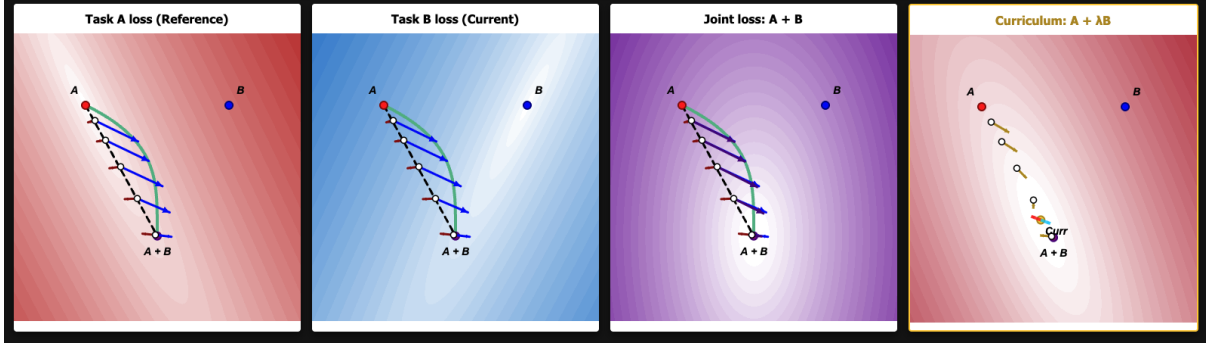


Figure 1.1: Conceptual visualisation of the optimisation landscape at a task transition, shown over a shared two-dimensional parameter space. Task A (red) and Task B (blue) are the single-task losses with minima at A and B ; the joint loss $A+B$ (purple) has its minimum at $A \cap B$. The dashed line is the straight interpolation from A to $A \cap B$; the green curve is the steepest-descent trajectory of the joint loss, which bows away from Task A’s basin and induces the stability gap. The fourth panel shows the λ -curriculum loss $A + \lambda B$: a small weight λ on Task B reshapes the basin into a valley that runs continuously from A toward the joint minimum, so the optimum moves along the corridor instead of teleporting across it. Arrows are normalised to a fixed length and show gradient direction only.

Because the optimum now moves continuously, the curriculum addresses two of the three contributors at once. It removes the trajectory effect, since the descent no longer bows away from Task A’s basin (the green curve), and it tempers the magnitude imbalance, since the large new-task gradient is introduced gradually rather than dominating the first step. Only the third contributor, the variance of the finite-buffer estimator, is independent of the schedule and left untouched. Treating the task-boundary discontinuity as the object to be fixed is the central contribution of this thesis; Section 4 makes the argument precise.

Beyond the basic linear ramp, we develop three refinements, each targeting a limitation of the schedule: an **adaptive schedule** that removes the need to choose the ramp length N , a $\lambda_{\min}$ **floor** that prevents the schedule from stalling progress on the new task, and **momentum** that suppresses the per-step accuracy oscillation the continuous-limit argument abstracts away. Section 6 describes how each is constructed and why.

The remainder of this thesis develops these ideas. Section 2 reviews continual learning, Experience Replay, the path-finding methods we compare against, the stability-gap literature, and two lines of work, blurry task boundaries and mode connectivity, that motivate the curriculum. Section 3

then sets out the thesis’s own account of the gap: it decomposes the gap into three contributors and contrasts path-finding with landscape-shaping. Section 4 proves that a continuous linear λ -ramp removes the trajectory contributor, with a residual of order $1/N$ in the ramp length. Section 5 states the research questions, and Section 6 describes the experimental testbed and the conditions used to test each prediction.

2 Background and Related Work

2.1 Continual Learning

To address catastrophic forgetting and the stability–plasticity dilemma, researchers have proposed a variety of methods. van de Ven et al. [22] taxonomise these methods into six computational mechanisms. *Replay* stores past examples in a buffer and interpolates them with new-task data, optimising the joint loss of the two. *Parameter regularisation* adds a penalty that anchors parameters identified as important for past tasks near their previous values, where importance is estimated from a curvature approximation of the loss [13]. *Functional regularisation* instead penalises changes to

the network’s input–output mapping at a set of anchor points, typically through a distillation loss. *Optimisation-based* methods modify stochastic gradient descent itself, seeking wider minima, adapting per-parameter learning rates, or projecting gradients to avoid interference. *Context-dependent processing* segregates tasks by routing computation per context, either by expanding the network or by gating parameters. *Template-based classification* sidesteps class-incremental decision boundaries by classifying against learned class prototypes, often in a frozen embedding space, rather than a trained softmax head. This thesis studies the replay family, and in particular its simplest member, Experience Replay.

2.2 Experience Replay

Experience Replay (ER) is the simplest replay baseline [4]. At each step it draws a replay mini-batch from the buffer, combines it with the current-task mini-batch, and takes a single optimiser step on the summed loss of the two. The buffer is populated by reservoir sampling [23], so every sample seen so far occupies a slot with equal probability regardless of arrival order. During T_0 the buffer is empty by construction, so ER reduces to plain SGD on the first task, and the stability gap is measurable only from T_1 onward.

This simplicity is why ER remains competitive. As Chaudhry et al. [4] show, jointly training with a small episodic memory often outperforms purpose-built methods, scaling gracefully to longer sequences without sacrificing plasticity like heavy regularisation does. Furthermore, because ER avoids explicitly manipulating gradients, it provides a clean setting to isolate the causes of the stability gap. We therefore adopt it as one of our two core baselines.

A finite buffer is an imperfect estimator of the true past-task distribution. Aljundi et al. [1] frame buffer selection as a constraint-reduction problem in which the stored samples approximate the feasibility region defined by the full past data; reservoir sampling is one (non-optimal) policy within that frame. The replay gradient computed on a small buffer therefore differs in direction and magnitude from the gradient on all past data, a discrepancy present at every step, independent of any task-boundary schedule.

2.3 Path-Finding Methods

A second class of replay-based and replay-adjacent methods constrains the SGD update at the task boundary using information about the past tasks. They differ in what information that is. GEM stores past-task gradients and projects the new-task gradient onto the nearest direction whose inner product with every stored gradient is non-negative, so the update does not increase the loss on any past task in the buffer [16]; this uses first-order (gradient) information only. A-GEM replaces the per-task constraints with a single averaged past-task gradient, reducing the projection to one cheap operation [3]. Both act only on the current step, not across the trajectory. EWC instead uses second-order information: it adds a quadratic penalty around the past optimum weighted by the diagonal of the Fisher information, slowing changes to parameters the old task constrained [13]. NCL also draws on curvature, preconditioning the update by the precision matrix of the past-task posterior to take a natural-gradient step along directions that leave the replay loss roughly unchanged [12]. In the framing of this thesis (Section 3.2), all four are *path-finding* methods: they accept the discontinuity at the task boundary and use stored gradient or curvature information to pick a safe trajectory across it. NCL is the second baseline we compare against.

2.4 The Stability Gap

Lange et al. [15] identified the stability gap using continual evaluation, measuring past-task accuracy after every update rather than only at task boundaries. The gap is a temporary, sharp drop in past-task accuracy at the start of a new task, from which the model only partially recovers. This drop is most severe during the first few hundred steps. While larger replay buffers help reduce it, the gap survives reservoir sampling and moderate learning rates, and the minimum accuracy always falls below the final recovered baseline.

The literature traces this gap to two compound contributors. The first is a severe gradient imbalance at the task boundary: because the gradient is dominated by the new task, the optimiser’s initial steps prioritize plasticity and directly sacrifice old-task stability [15]. The second contributor is the underlying geometry of the descent path, which

proves the gap is not just caused by imperfect memory. Hess et al. [10] show that the gap persists even when training on the exact joint loss with full access to all past data. Kao et al. [12] demonstrate this analytically: even with perfect joint gradients, steepest descent from an old-task minimum inherently forces the trajectory out of the old basin. Together, these results separate *what* is optimised from *how* it is optimised. A perfect objective function does not prevent the gap; the geometric trajectory (visualised in Figure 1.1) must be addressed directly.

The gap is broad rather than incidental: it appears in joint incremental learning of homogeneous tasks [11], in continual pre-training of large language models [8], and in relation to the classification head [24], and mitigations have been proposed, from temporal ensembles [21] to explicit gap-reduction procedures [9].

2.5 Blurry Task Boundaries

A parallel line of work relaxes the assumption that tasks arrive at sharp boundaries, replacing the instantaneous switch from T_0 to T_1 with a gradual change. The *blurry* setting, introduced with Rainbow Memory [2], models this overlap explicitly. Classes from neighbouring tasks coexist in time, meaning samples of an upcoming class appear before the previous task ends. As a result, each task shades smoothly into the next rather than abruptly replacing it. In this setting, the replay buffer’s composition matters more than in disjoint streams because it must represent classes that are still arriving. Rainbow Memory addresses this by selecting stored samples for their *diversity* using an uncertainty-based scoring method. Combined with data augmentation, this approach outperforms standard disjoint-buffer baselines by a wide margin.

Online formulations push this boundary-free idea down to the level of individual samples. Koh et al. [14] study a class-incremental blurry stream under an *anytime-inference* constraint. Because the model may be queried at any point during training, it must remain accurate throughout the transition, rather than merely converging well at the end. Continuous evaluation surfaces the transient accuracy drops that conventional end-of-task metrics hide. This motivates both a metric that integrates accuracy over the entire stream and memory

management designed to keep performance high at every step. In both settings, the core commonality is that the task transition spans many steps rather than occurring instantly.

This gradual transition is what connects these settings to our intervention. In a blurry stream, a mini-batch shifts progressively from almost entirely T_0 data to almost entirely T_1 data over the transition window. Consequently, the gradient the optimiser follows is a moving mixture that shifts gradually from the old task to the new. Our λ -curriculum produces this exact same moving mixture, but through the *loss weight* rather than the *data*. It scales the new-task loss term up from zero while holding the replay term fixed. Up to an overall scale, both methods descend the same objective. Both keep the past-task gradient present from the very first step while introducing the new-task gradient gradually.

2.6 Linear Mode Connectivity

The introduction claimed that a low-loss corridor connects the single-task solution to the joint optimum. Work on *mode connectivity* supports this. Garipov et al. [7] and Draxler et al. [5] showed that independently trained minima of a deep network, though separated by a barrier along the straight line between them, are joined by curved paths of near-constant low loss. Frankle et al. [6] sharpened this to *linear* mode connectivity: networks that share a short period of early training can be joined by a straight low-loss path. For continual learning specifically, Mirzadeh et al. [19] found that the multitask solution and the sequentially trained solution often lie in the same basin, linearly connected, when the training regime is stable. This matters for the stability gap in two ways. It means the dip is not forced by the geometry, because a low-loss route from θ_A^* to θ_{joint}^* exists; and it sharpens what an intervention has to do, namely keep the optimiser on that route rather than let it bow away, as the trajectory argument predicts it otherwise will.

3 A Discontinuity Account of the Stability Gap

The previous section surveyed three findings that each touch a different face of the same object:

the gap survives the joint-loss oracle (Section 2.4), gradual transitions forget less (Section 2.5), and a low-loss corridor demonstrably connects the single-task solution to the joint optimum (Section 2.6). None of that prior work names a single mechanism that ties them together. This section sets out the account this thesis defends: the stability gap is, in its dominant components, the downstream symptom of the discontinuity at the task boundary introduced in Section 1. We make the claim precise by decomposing the gap into three additive contributors (Section 3.1) and then by contrasting the two ways one can intervene on the discontinuity (Section 3.2).

3.1 Decomposing the Stability Gap

Previous work has identified various causes for the stability gap: the gradient imbalance (Section 2.4), the finite-buffer estimator (Section 2.2), and the trajectory effect (Section 2.4). Here, we unify these into a single decomposition, breaking the gap down into three distinct forces that act at different stages of training. We will isolate and measure each empirically in Section 6.

Consider the first step of vanilla Experience Replay (ER) at the start of a new task, T_1 . Because the optimal parameters for the previous task (θ_0^*) form a stationary point for the replay loss, the true replay gradient at this point is near zero. Consequently, the first parameter update is driven almost entirely by the new task:

$$\theta_1 \approx \theta_0^* - \eta(\varepsilon(\theta_0^*) + g_{\text{new}}(\theta_0^*)), \quad (3.1)$$

where $g_{\text{new}} := \nabla L_{\text{current}}$ and ε is the zero-mean error of the finite-buffer estimator. This initial step triggers three distinct phenomena, each contributing to the subsequent performance dip on T_0 :

Magnitude asymmetry (G_{mag}). At θ_0^* the new-task gradient g_{new} does not vanish — the new task is still unlearned — whereas the buffer sampling noise ε is comparatively small and shrinks as the buffer grows. The first step is therefore dominated by the new task, $-\eta g_{\text{new}}$, and carries the iterate out of the replay basin. This is the same mechanism as the trajectory dip of Section 2.4: any displacement away from θ_0^* raises the replay loss, so the dip appears at once — only here it is driven by

the unbalanced first step rather than an exact joint step. Because the imbalance is fixed by the relative gradient magnitudes at step one, this is the dominant contributor to the gap, and corresponds to the gradient-imbalance effect observed by Lange et al. [15].

Estimator variance (G_{est}). In any individual training run, $\varepsilon \neq 0$, which adds a stochastic kick to θ_1 . As discussed in Section 2.2, the replay buffer provides only a finite-sample estimate of the past data [1]. This per-step sampling noise creates a baseline loss degradation that persists regardless of the learning schedule, as it is independent of how the loss terms are composed.

Trajectory effect (G_{traj}). Even in an idealized scenario where magnitude asymmetry is corrected and buffer noise is eliminated ($\varepsilon \equiv 0$), a geometric penalty remains. As outlined in Section 2.4, an instantaneous switch in objectives forces the iterate off the optimal path of the joint loss. Along this new trajectory, the replay loss is strictly higher than it would be at the joint-loss optimum θ_{joint}^* . This is the structural contributor addressed by the envelope-theorem argument in Section 4.

Crucially, these three components dominate at different timescales: G_{mag} drives the massive disruption at step one, G_{traj} dictates the curve over the first equilibration window, and G_{est} provides a constant hum of noise across all steps.

3.2 Path-Finding versus Landscape-Shaping

The decomposition suggests two ways to intervene, distinguished by what they do with the discontinuity rather than by what curvature information they use. *Path-finding* accepts the discontinuity. The objective still jumps from L_{replay} to $L_{\text{replay}} + L_{\text{current}}$ at the boundary, and the intervention reshapes *how* the optimiser moves, replacing plain SGD with a preconditioned step $\theta_{t+1} = \theta_t - \eta M(\theta_t) \hat{g}_t$. Here M is built from past-task information — the inverse Fisher of EWC [13], the precision matrix of NCL [12], or the feasible-set projection of GEM [16]. The path-finding methods of Section 2.3 all sit here.

Landscape-shaping removes the discontinuity. It modifies the objective in a neighbourhood of

the boundary so the surface the optimiser descends is continuous through the transition, leaving the geometry of the step to SGD’s natural step-size adaptation. The λ -curriculum is the canonical example: the gradient followed is $\hat{g}_t = \nabla(L_{\text{replay}} + \lambda(t)L_{\text{current}})(\theta_t)$ with $\lambda(t)$ a continuous schedule from 0 to 1. Two consequences distinguish it from path-finding. A preconditioner that normalises the gradient breaks SGD’s step-size adaptation, whereas $\lambda(t)$ multiplies the loss and flows through the backward pass to every parameter without rescaling the step; and the preconditioned step needs a per-step matrix–vector product, whereas the curriculum adds one scalar multiply. Section 4 shows that this continuity is enough to remove the trajectory contributor G_{traj} , and that it simultaneously tempers G_{mag} by introducing g_{new} gradually rather than all at once.

4 The Envelope-Theorem Bound: Why a Linear λ -Ramp Works

When a model switches instantly from an old task to a new one (λ jumps from 0 to 1), the sudden change causes the optimizer to “cut the corner” and veer far off the ideal learning path. This violent detour causes a massive, temporary spike in forgetting, which Section 3.1 identified as the trajectory gap (G_{traj}).

This section outlines why replacing that sudden jump with a slow, continuous transition (a linear ramp over N steps) mathematically eliminates this spike. The full, step-by-step mathematical derivation of this proof is provided in Appendix 6.7.

The core argument is built on two intuitive pillars:

4.1 Part A: The perfect path never overshoots

Imagine an “optimum path” ($\Theta^*(\lambda)$) that perfectly tracks the exact bottom of the combined loss valley as the curriculum transitions from pure replay ($\lambda = 0$) to the joint task ($\lambda = 1$). Figure 4.1 shows this path directly: as λ grows, the minimiser of the curriculum loss $L_\lambda = A + \lambda B$ migrates smoothly from the old-task optimum A to the joint optimum

$A+B$, so a model that follows it rides a moving valley floor rather than jumping between the two.

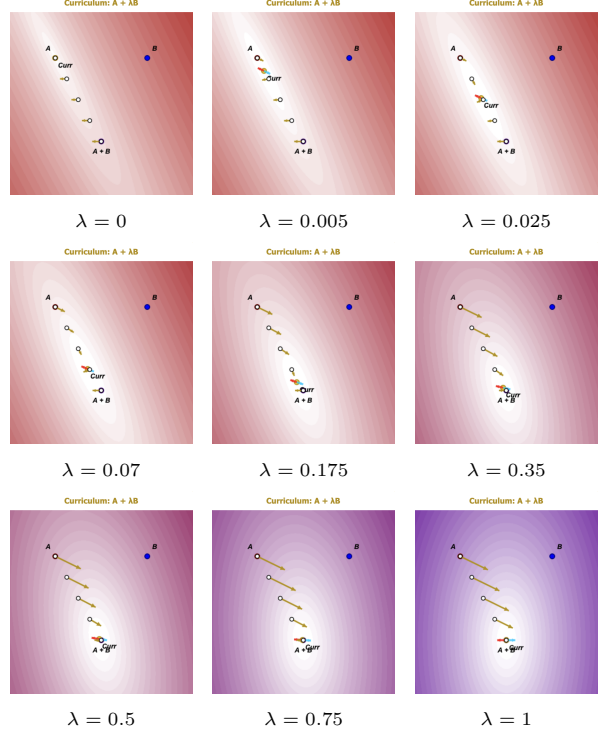


Figure 4.1: Evolution of the curriculum loss $L_\lambda = A + \lambda B$ over a shared two-dimensional parameter space. Read left to right, top to bottom, the weight on the new task ramps from $\lambda = 0$ (pure replay) to $\lambda = 1$ (the joint loss); the value of λ is printed beneath each panel. A and B mark the two single-task minima and $A+B$ the joint minimum. As λ increases the contours morph continuously from the old-task basin around A to the joint basin around $A+B$, and the current minimiser ($Curr$) slides along the gold chain from A toward $A+B$. That chain is the optimum path $\Theta^*(\lambda)$ of Section 4.1. The opposing red and blue arrows at the current minimiser are the replay- and current-task gradient contributions, which cancel there by the first-order condition $\nabla L_{\text{replay}} + \lambda \nabla L_{\text{current}} = 0$. Because the minimiser never jumps — it only drifts as λ grows — a model that tracks it rides the moving valley floor instead of being teleported across the landscape.

If the model were capable of perfectly walking this ideal path, the replay loss would never experience a sudden spike. As proven in the Appendix

using the envelope theorem, the rate of change of the replay loss along this path is mathematically guaranteed to be non-negative:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0 \quad \text{for all } \lambda \in [0, 1]. \quad (4.1)$$

Because the replay loss can only strictly rise (or stay flat) as it accommodates the new task, it can never decrease. Therefore, it is physically impossible for the loss to spike up and come back down. It simply rises monotonically until it reaches its permanent, unavoidable baseline at the joint optimum (θ_{joint}^*).

4.2 Part B: A slow ramp keeps the model on the path

Part A describes a mathematically perfect path, but real SGD iterates ($\theta(t)$) are clumsy and take discrete steps. However, by analyzing the gradient flow, we can prove that the actual model trajectory acts as though it is tethered to the perfect path.

The distance the model lags behind the perfect path is directly proportional to how fast the valley floor is moving. If the transition is stretched over a sufficiently long ramp (large N), the maximum distance the model wanders off the ideal path is tightly bounded:

$$\|\theta(t) - \Theta^*(\lambda(t))\| = O\left(\frac{1}{N}\right). \quad (4.2)$$

4.3 The Combined Bound

By combining the "no-overshoot" rule of the perfect path (Part A) with the tracking error of the actual network (Part B), we arrive at the final bound for the highest replay loss the model will experience during the transition:

$$\max_t L_{\text{replay}}(\theta(t)) \leq L_{\text{replay}}(\theta_{\text{joint}}^*) + O\left(\frac{1}{N}\right). \quad (4.3)$$

The first term is the genuine plasticity–stability trade-off: the permanent capacity cost of learning both tasks. The second term is the transient stability gap (G_{traj}).

Because the transient gap shrinks proportionally with $1/N$, a sufficiently slow continuous curriculum physically drives the trajectory overshoot to zero.

The model’s step-size adaptation is left untouched, cleanly resolving the gap without the need for complex path-finding interventions.

5 Research Questions

The decomposition of Section 3.1 and the bound of Section 4 make a set of falsifiable predictions about how the λ -curriculum and its refinements should behave. We evaluate them on rotated MNIST with a small multilayer perceptron, the regime in which the envelope-theorem bound is sharply testable, using vanilla and gradient-balanced Experience Replay as baselines. Five research questions structure the experiments.

RQ1 (linear curriculum). Does the linear λ -curriculum applied to standard ER monotonically reduce stability-gap depth as the ramp length N grows, and does it do so without the final-accuracy cost incurred by gradient balancing?

RQ2 (adaptive curriculum). Does an adaptive λ -schedule, which sets λ from the live ratio of replay-to-current gradient magnitudes and so removes the explicit knob N , reduce gap depth without that final-accuracy cost?

RQ3 (momentum). Does adding momentum on top of the curriculum reduce the residual depth fluctuation predicted by the discrete-time stochastic argument? And does momentum on its own, without the curriculum, produce any comparable benefit — the prediction being that it does not, since the one-sided first step is deterministic?

RQ4 ($\lambda_{\min}$). Does a small floor $\lambda_{\min} > 0$ on the adaptive schedule recover early-task learning speed without re-introducing the depth pathology the curriculum was designed to remove?

RQ5 (decomposition). How does the gap depth decompose into the magnitude, estimator, and trajectory contributions of Section 3.1, and what does each share reveal about why a curriculum is the right shape of fix?

Section 6 describes the testbed, the metrics used to measure gap depth and final accuracy, and the

full set of conditions that isolate each contributor and test each prediction.

6 Methods

This section describes the empirical testbed (Section 6.1), the metrics that summarise each run (Section 6.2), and the full set of conditions (Sections 6.3–6.7) that test the predictions of Sections 3 and 4. Each experimental block maps to one of the research questions of Section 5 and to a falsifiable prediction derived earlier.

6.1 Testbed

6.1.1 Dataset: Rotated MNIST

The primary sweep uses rotated MNIST (rot-MNIST) as a two-task sequence: the first task T_0 presents the digits at 0° and the second task T_1 presents the same digits rotated by 90° . Each task uses the standard 60,000 training and 10,000 test images.

The 90° rotation makes the two tasks visually distinct — their input distributions are well separated — while keeping them structurally identical, with the same label space and the same underlying digit identities. This makes the gradient dynamics at the transition maximally informative: the tasks differ enough that the new-task gradient is large at θ_0^* , yet are similar enough that the replay buffer remains directly representative of the past data distribution. Two tasks produce exactly one transition, and so exactly one instance of the stability gap, which keeps the analysis clean: there is no interference between several overlapping gaps.

6.1.2 Model: MLP

The backbone for the rot-MNIST sweep is a two-layer multilayer perceptron (MLP) with ReLU activations (Table 6.1; full details in Appendix ??).

Component	Value
Input	784 (flattened 28×28 greyscale)
Hidden layers	2×400 (fully connected, ReLU)
Output	10-class logits (no activation)
Regularisation	none (no dropout, no batch norm)
Total parameters	478,410

Table 6.1: MLP backbone used for the rot-MNIST sweep.

The MLP is chosen over a deeper network for the rot-MNIST sweep on deliberate grounds. It has a simpler loss landscape, with fewer spurious curvature effects unrelated to the curriculum dynamics; it trains in seconds, which matters because the sweep spans many conditions and seeds; and it stays close to the adiabatic regime in which the Hessian assumptions behind the envelope-theorem bound of Section 4 hold approximately, so that bound is sharply testable. A ResNet-18 backbone is used for the CIFAR-10 generalisation experiment of Section 6.7; details are in Appendix ??.

6.1.3 Training protocol

Table 6.2 lists the training hyperparameters. The online regime of one epoch per task is standard for stability-gap analysis: it gives a single clean pass through the transition, where the gap is most pronounced.

Hyperparameter	Value
Epochs per task	1 (online regime)
Batch size	256
Optimiser	SGD
Learning rate	0.1
Momentum	0.0 default; 0.9 for momentum conditions
Weight decay	0.0
Replay buffer (small)	1,000 samples, reservoir sampling
Replay buffer (full)	60,000 samples (all past data)
Seeds	5 per condition

Table 6.2: Training protocol for the rot-MNIST sweep. The full-data buffer is used only for the full-data decomposition conditions of Section 6.4.

6.1.4 Evaluation protocol

Dense per-step evaluation is used during the first 500 steps of T_1 training. A stability-gap tracker snapshots the pre-task accuracy on the T_0 test set

before T_1 begins, evaluates the model on the T_0 test set at every training step, and records the resulting (step, accuracy) series. After T_1 training it computes the scalar summaries of Section 6.2.1. A gradient tracker records, at the same per-step cadence, the directional and magnitude fidelity of the replay-buffer gradient against the deterministic empirical past-task gradient g_{true} (Section 6.2.3).

Periodic evaluation, every 10 steps, is used for the general training-loss and final-accuracy curves outside the boundary window. The CIFAR-10 block of Section 6.7 coarsens the dense cadence to every 50 steps over a 250-step window: at ten epochs per task the post-switch dynamics play out over thousands of steps, so a step-1 cadence over a 500-step window would no longer cover the gap and would inflate logging cost on the deeper backbone.

6.2 Metrics

All metrics are recorded automatically by the evaluation pipeline and reported as mean $\pm$ standard deviation across the five seeds.

6.2.1 Stability-gap metrics

These are computed from the dense per-step evaluation. Write a_j^{pre} for the pre-task accuracy on task j and $a_j(t)$ for its accuracy at step t of the new task.

6.2.2 Continual-learning metrics

Following Lange et al. [15], we also report the continual-evaluation metrics in Table 6.4, computed at task boundaries and over the full record. Let $A(E_j, f_n)$ be the accuracy on the test set of task j evaluated at the model f_n after training stage n , and let N be the number of tasks. The full task-boundary accuracy matrix $R[i, j] = A(E_j, f_{T_i})$ is logged for every run, so any downstream metric can be reconstructed.

6.2.3 Gradient-fidelity diagnostics

The gradient tracker compares the replay-buffer gradient g_{replay} against the true past-task gradient g_{true} , computed at every step by a separate forward and backward pass over the entire past-task

training set (no sub-sampling, so g_{true} is the deterministic empirical past-task gradient at the current parameters). Table 6.5 lists the diagnostics.

The first two diagnostics carry the **true_grad** prefix because they are measured against the true full-data past-task gradient g_{true} : they quantify how faithfully a finite buffer approximates the past data the gradient is meant to represent [1]. The third, **grad_ratio**, compares g_{replay} to the current-task gradient g_{new} and is unrelated to g_{true} ; its inverse $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$ is the signal the adaptive curriculum of Section 6.5 uses to set $\lambda(t)$.

The buffer-fidelity diagnostics are enabled for the decomposition conditions only (Section 6.4), where they give direct per-step evidence for the estimator-bias contributor and pair with the indirect evidence from the depth difference between balanced and full-data conditions. Each step adds one forward and backward pass over every past-task training sample (60,000 for rot-MNIST T_0), which is affordable on the MLP testbed but would dominate runtime on larger backbones, so the toggle stays off for the curriculum blocks, whose claims rest on **gap_depth** and ACC rather than per-step gradient geometry.

6.2.4 Statistical comparison

Across conditions we compare metrics with the paired Wilcoxon signed-rank test on the seed-paired difference. Seeds are paired by index across conditions, so the per-seed deltas $\Delta_s = m(C_a, s) - m(C_b, s)$ factor out the seed-induced variance that would otherwise dominate the five-seed sample. We prefer Wilcoxon over the paired t -test because **gap_depth** and accuracy are bounded in $[0, 1]$ and typically right-skewed near zero, so the t -test’s normality assumption is suspect; the Wilcoxon test exchanges that assumption for the milder one that Δ_s is distributionally symmetric under the null. For equivalence-style claims, such as “the adaptive curriculum matches the best linear curriculum”, we report the seed-paired difference with its bootstrap 95% confidence interval rather than a null-hypothesis test.

6.3 Overview of Experiments

Each block tests one or more predictions of Sections 3 and 4 and answers one research question

Metric	Definition	Role
gap_depth	$\max_j (a_j^{\text{pre}} - \min_t a_j(t))$ over the full record window	Primary metric — worst single point of instability
gap_area	$\sum_j \int \max(0, a_j^{\text{pre}} - a_j(t)) dt$ (trapezoidal)	Captures depth $\times$ duration in one scalar
max_drop	gap_depth restricted to the first 200 steps	Fixed-window metric, kept for back-comparability
recovery_steps	First step at which every past task is at $\geq 90\%$ of its pre-task baseline, measured from the point of maximum drop	None if no recovery is observed in the window

Table 6.3: Stability-gap metrics, recorded from dense per-step evaluation. `gap_depth` is the headline measure used in every contrast.

Metric	Definition	Role
ACC	$\frac{1}{N} \sum_j A(E_j, f_{T_{N-1}})$	Average task accuracy at the final model
FORG	$\frac{1}{N-1} \sum_{i < N-1} [A(E_i, f_{T_i}) - A(E_i, f_{T_{N-1}})]$	Average drop from right-after-learning to final
min-ACC	$\frac{1}{N-1} \sum_{i < N-1} \min\{A(E_i, f_n) : n > T_i\}$	Worst-case retention since learning
WF _w	Mean over tasks of the largest accuracy drop in any window of w consecutive evaluations ($w \in \{10, 100\}$)	Worst-case stability over short / medium horizons
WP _w	Symmetric counterpart of WF _w : the largest accuracy gain ($w \in \{10, 100\}$)	Plasticity / learning velocity
WC-ACC	$\frac{1}{N} A(E_{N-1}, f_{T_{N-1}}) + (1 - \frac{1}{N}) \text{min-ACC}$	Worst-case trade-off — a lower bound on ACC

Table 6.4: Continual-learning metrics of Lange et al. [15], recorded at task boundaries and over the full record.

Diagnostic	Logged name	Role
$\cos(g_{\text{replay}}, g_{\text{true}})$	true_grad_cosine	Buffer-side directional fidelity — the estimator-bias contributor of Section 6.4
$\frac{\ g_{\text{replay}}\ }{\ g_{\text{true}}\ }$	true_grad_mag_ratio	Buffer-side magnitude fidelity
$\frac{\ g_{\text{new}}\ }{\ g_{\text{replay}}\ }$	grad_ratio	Update-side magnitude asymmetry — large at step 1 when $\ g_{\text{replay}}\ \approx 0$

Table 6.5: Gradient-fidelity diagnostics, recorded over the new task’s first ~ 500 steps. Each is logged as a per-step series together with its mean (and minimum, for the cosine).

(Table 6.6). All conditions use five seeds and the training protocol of Section 6.1 unless noted otherwise.

6.4 Three-Contributor Decomposition

Aim. Isolate the three contributors of Section 3.1 — magnitude asymmetry (G_{mag}), estimator variance (G_{est}), and the trajectory effect (G_{traj}) — and quantify each one’s share of the total gap. This is the diagnostic question of RQ5.

Rationale. Two of the three contributors are predicted to be downstream symptoms of the discontinuity at the task boundary; the third, estimator variance, is a property of finite replay buffers and persists at any schedule. Removing each contributor by construction and observing the residual gap depth identifies which contributors drive the dip. The path $G1 \rightarrow G2 \rightarrow G3 \rightarrow G4$ removes one source at a time (Table 6.7).

Full-data conditions. The full-data conditions G2 and G4 hold the entire past-task training set in the buffer and, at every step, compute the replay gradient on every stored sample exactly once. The resulting replay gradient is the deterministic empirical past-task gradient at the current parameters — no sampling, no bootstrap variance. Vanilla ER (G1) draws 256 samples with replacement, so its replay gradient is a stochastic estimator of this same quantity. The estimator contributor G_{est} is therefore exactly the gap-depth difference between an estimator-driven step and an exact-gradient step, and G_{traj} in G4 is the residual once both the estimator and the magnitude asymmetry are removed.

Decomposition shares. Differences along the path identify each contributor:

$$G_{\text{mag}} := \text{gap_depth}(G1) - \text{gap_depth}(G3), \quad (6.1)$$

$$G_{\text{est}} := \text{gap_depth}(G3) - \text{gap_depth}(G4), \quad (6.2)$$

$$G_{\text{traj}} := \text{gap_depth}(G4). \quad (6.3)$$

G_{mag} is the share removed by balancing alone; G_{est} is the additional share removed by full data on top of balancing; G_{traj} is the residual after both are removed, the trajectory contribution of

Section 4. The G2 condition is a sanity check: $\text{gap_depth}(G1) - \text{gap_depth}(G2)$ should recover G_{est} along an orthogonal path through the design.

NCL reference. NCL is included as a representative path-finding method (Section 3.2): it accepts the discontinuity but preconditions each SGD step by the past-task precision matrix. Its gap depth is the natural reference for how much depth reduction is achievable by acting in parameter space rather than in schedule space, and gives a fair comparison point for the λ -curriculum.

Expectation. Gap depth reduces along the path $G1 \rightarrow G3 \rightarrow G4$. The trajectory residual at G4 is small but non-zero (the prediction of Kao et al. [12], Section 2.4), and is the contributor the λ -curriculum should additionally suppress.

6.5 The λ -Curriculum Sweep

Aim. Test the envelope-theorem prediction of Section 4 — that a continuous λ -schedule eliminates the trajectory contribution to the gap, with a transient that shrinks as $O(1/N)$ (Equation (4.3)) — and the $\lambda_{\min}$ trade-off. Answers RQ1, RQ2, and RQ4.

Conditions. All curriculum conditions are applied on top of standard ER (1k reservoir, no balancing, no NCL) so the curriculum’s effect is isolated (Table 6.8).

The adaptive schedule is self-paced. At θ_0^* the replay gradient is near zero by stationarity, so the ratio $r := \|g_{\text{replay}}\|/\|g_{\text{new}}\|$ is near zero and λ starts near 0. As the iterate leaves θ_0^* , $\|g_{\text{replay}}\|$ grows while $\|g_{\text{new}}\|$ shrinks as the model fits T_1 , so r climbs and $\lambda \rightarrow 1$ with no explicit ramp length; the schedule ignores N entirely. The floor $\lambda_{\min}$ matters mainly for this adaptive variant: it stops the schedule from sitting at $\lambda \approx 0$ in the first steps, when the EMA estimate of r is still close to zero.

Expectation.

- *Linear sweep* (RQ1). gap_depth should fall monotonically as N grows from 50 to 200, consistent with the $O(1/N)$ bound of Equation (4.3). ACC should stay close to vanilla ER, because the curriculum preserves SGD’s

Block	Section	RQ	Falsifiable prediction	Conditions
Three-contributor decomposition	6.4	RQ5	Magnitude, estimator, and trajectory together account for the total gap; the trajectory residual is small but non-zero	G1–G4, plus NCL reference
Linear λ -curriculum	6.5	RQ1	<code>gap_depth</code> falls monotonically with N ; ACC matches vanilla ER	$N \in \{50, 100, 200\}$
Adaptive λ -curriculum	6.5	RQ2	<code>gap_depth</code> falls relative to vanilla ER with no explicit knob N	adaptive schedule from $\ g_{\text{replay}}\ /\ g_{\text{new}}\ $
$\lambda_{\min}$ refinement	6.5	RQ4	A floor $\lambda_{\min} > 0$ recovers early velocity without inflating depth to vanilla-ER levels	$\lambda_{\min} \in \{0.05, 0.10, 0.20\}$ on adaptive
Momentum cross-cut	6.6	RQ3	Momentum tightens depth on top of the curriculum, but momentum alone does not reduce depth	every rot-MNIST condition repeated at momentum 0.9
CIFAR-10 generalisation	6.7	all	Headline contrasts carry to a stronger task shift and a deeper backbone	vanilla ER, NCL, adaptive $\lambda_{\min}=0.20$, adaptive $\lambda_{\min}=0.10$

Table 6.6: Overview of the experimental blocks, the research question each answers, and the prediction each tests.

Condition	Replay buffer	Gradient handling		Removed by construction	Residual gap
G1 — Vanilla ER	1k reservoir	unbalanced		nothing	$G_{\text{mag}} + G_{\text{est}} + G_{\text{traj}}$
G2 — Full-data ER	60k (all past)	unbalanced		estimator noise (exact gradient)	$G_{\text{mag}} + G_{\text{traj}}$
G3 — Balanced ER	1k reservoir	unit-norm	balanced	magnitude asymmetry	$G_{\text{est}} + G_{\text{traj}}$ (+ balancing cost)
G4 — Full-data + balanced	60k	unit-norm	balanced	magnitude and estimator	G_{traj}
NCL (reference)	none	K-FAC	precision preconditioning	—	path-finding reference [12]

Table 6.7: Decomposition conditions. Each row removes one source of the gap relative to G1, so the residual gap shrinks down the path G1 → G3 → G4.

Condition	$\lambda(t)$ schedule	Hyperparameter	Tests
Vanilla ER (baseline)	$\lambda \equiv 1$ from step 1 (discontinuous)	—	reference
Linear $N=50$	$\text{clip}(t/50, 0, 1)$	$N = 50$	RQ1
Linear $N=100$	$\text{clip}(t/100, 0, 1)$	$N = 100$	RQ1
Linear $N=200$	$\text{clip}(t/200, 0, 1)$	$N = 200$	RQ1
Adaptive	$\text{clip}(\text{EMA}_{\alpha}(\ g_{\text{replay}}\ /\ g_{\text{new}}\), 0, 1)$	$\alpha = 0.05$	RQ2
Adaptive + $\lambda_{\min}$	$\max(\lambda_{\min}, \text{clip}(\text{EMA}_{\alpha}(\ g_{\text{replay}}\ /\ g_{\text{new}}\), 0, 1))$	$\alpha = 0.05, \lambda_{\min} \in \{0.05, 0.10, 0.20\}$	RQ4

Table 6.8: λ -curriculum conditions, all applied on top of standard ER. EMA_{α} is an exponential moving average with smoothing α .

natural step-size adaptation rather than rescaling the step the way gradient balancing does.

- *Adaptive* (RQ2). The adaptive schedule should achieve a depth reduction comparable to a tuned linear curriculum, removing the need to fix N as a hyperparameter.
- $\lambda_{\min}$ (RQ4). Raising $\lambda_{\min}$ from 0 should lift the gap floor slightly — the trajectory cannot dip below the replay loss at the floor minimiser — while accelerating T_1 learning. We expect a small $\lambda_{\min} \in [0.05, 0.15]$ to give a better Pareto position on the (depth, ACC) trade-off than $\lambda_{\min} = 0$.

The contrast between balanced ER (G3) and linear $N=200$ isolates the difference between acting on the magnitude symptom alone and acting on the schedule discontinuity: both should suppress depth, but only the curriculum should preserve ACC.

6.6 Momentum and the Curriculum $\times$ Momentum Cross

Aim. Test the two momentum predictions behind RQ3: that momentum on top of the curriculum suppresses the residual stochastic oscillation around the optimum path, and that momentum alone, with no curriculum, does not reduce gap depth, because the one-sided first step at the boundary is deterministic and its direction is unchanged.

Design. Every rot-MNIST condition of Sections 6.4 and 6.5 is repeated with momentum 0.9 (Nesterov off), giving a $2 \times$ (decomposition + curriculum) cross (Table 6.9). The CIFAR-10 block of Section 6.7 ships a single momentum setting (0.9, the better-performing leg from rot-MNIST) and is not part of this cross.

Condition	Mom. 0.0	Mom. 0.9
Vanilla ER (G1)	✓	✓
Full-data ER (G2)	✓	✓
Balanced ER (G3)	✓	✓
Full-data + balanced (G4)	✓	✓
NCL	✓	✓
Linear $N \in \{50, 100, 200\}$	✓	✓
Adaptive	✓	✓
Adaptive + $\lambda_{\min} \in \{0.05, 0.10, 0.20\}$	✓	✓

Table 6.9: Momentum cross. Every rot-MNIST condition is run at momentum 0.0 and 0.9.

Expectation. On top of the curriculum, momentum should reduce the seed-to-seed standard deviation in **gap_depth** and the per-step oscillation amplitude on the T_0 accuracy curve, without changing the depth mean by much. Without the curriculum, momentum should not reduce **gap_depth**: depth for vanilla ER at momentum 0.9 should match depth at momentum 0.0. A null result here is the falsifiable prediction. The contrast (linear $N=200$, momentum on) versus (linear $N=200$, momentum off) measures the residual momentum cleans up; the contrast (vanilla ER, momentum on) versus (vanilla ER, momentum off) tests the falsification.

6.7 Generalisation: CIFAR-10

Aim. Confirm the direction of the rot-MNIST findings under a stronger task shift and a deeper, harder-to-analyse architecture — the regime in which the assumptions behind the envelope-theorem bound weaken (Section 4).

Dataset. A two-task domain-shift CIFAR-10: T_0 uses clean CIFAR-10 and T_1 uses the Gaussian-noise corruption at severity 3. Both tasks share the same ten classes, so only the input distribution shifts. As in rot-MNIST, two tasks produce exactly one transition.

Training protocol. A CIFAR-adapted ResNet-18 (3×3 stem, no max-pool stem; see Appendix ??), ten epochs per task — at this depth ResNet-18 needs more than one pass to converge before the transition — batch size 256, SGD with learning rate 0.1, and momentum 0.9 only. The momentum cross of Section 6.6 is not repeated; momen-

tum 0.9 is locked in as the better-performing leg from rot-MNIST. The dense evaluation cadence is coarsened to every 50 steps over a 250-step window (Section 6.1.4).

Conditions. Four headline conditions only, not the full sweep (Table 6.10). A linear curriculum is not run on CIFAR; only the two best-performing adaptive variants from the rot-MNIST $\lambda_{\min}$ refinement are carried over, alongside the two reference points. NCL hyperparameters are pinned to the values that won the rot-MNIST tuning sweep (Appendix ??).

Condition	Method	Tests
D1 — Vanilla ER	ER (1k reservoir)	reference
D2 — NCL	NCL (tuned)	reference
D3 — Adaptive, $\lambda_{\min}=0.20$	ER + adaptive curriculum	path-finding RQ4 carry-over
D4 — Adaptive, $\lambda_{\min}=0.10$	ER + adaptive curriculum	RQ4 carry-over

Table 6.10: CIFAR-10 headline conditions. Only the two best adaptive variants from rot-MNIST are carried over, with vanilla ER and NCL as references.

Expectation. Qualitative carry-over of the rot-MNIST result: the adaptive curriculum should reduce gap depth relative to vanilla ER and at least match NCL on depth while preserving ACC. The quantitative $O(1/N)$ scaling of the bound is not expected to hold tightly at this depth; deviations are reported and discussed alongside the results.

References

- [1] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning, 2019. URL <https://arxiv.org/abs/1903.08671>. 4, 6, 10
- [2] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, and Jonghyun Choi. Rainbow memory: Continual learning with a memory of diverse samples, 2021. URL <https://arxiv.org/abs/2103.17230>. 5
- [3] Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-gem, 2019. URL <https://arxiv.org/abs/1812.00420>. 2, 4
- [4] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc’Aurelio Ranzato. On tiny episodic memories in continual learning, 2019. URL <https://arxiv.org/abs/1902.10486>. 4
- [5] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht. Essentially no barriers in neural network energy landscape, 2019. URL <https://arxiv.org/abs/1803.00885>. 2, 5
- [6] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis, 2020. URL <https://arxiv.org/abs/1912.05671>. 5
- [7] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns, 2018. URL <https://arxiv.org/abs/1802.10026>. 2, 5
- [8] Yiduo Guo, Jie Fu, Huishuai Zhang, Dongyan Zhao, and Yikang Shen. Efficient continual pre-training by mitigating the stability gap, 2024. URL <https://arxiv.org/abs/2406.14833>. 5
- [9] Md Yousuf Harun and Christopher Kanan. Overcoming the stability gap in continual learning, 2024. URL <https://arxiv.org/abs/2306.01904>. 5
- [10] Timm Hess, Tinne Tuytelaars, and Gido M. van de Ven. Two complementary perspectives to continual learning: Ask not only what to optimize, but also how, 2024. URL <https://arxiv.org/abs/2311.04898>. 2, 5

- [11] Sandesh Kamath, Albin Soutif-Cormerais, Joost van de Weijer, and Bogdan Raducanu. The expanding scope of the stability gap: Unveiling its presence in joint incremental learning of homogeneous tasks, 2024. URL <https://arxiv.org/abs/2406.05114>. 5
- [12] Ta-Chu Kao, Kristopher T. Jensen, Gido M. van de Ven, Alberto Bernacchia, and Guillaume Hennequin. Natural continual learning: success is a journey, not (just) a destination, 2021. URL <https://arxiv.org/abs/2106.08085>. 2, 4, 5, 6, 12, 13
- [13] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dhharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. ISSN 1091-6490. doi: 10.1073/pnas.1611835114. URL <http://dx.doi.org/10.1073/pnas.1611835114>. 2, 3, 4, 6
- [14] Hyunseo Koh, Dahyun Kim, Jung-Woo Ha, and Jonghyun Choi. Online continual learning on class incremental blurry task configuration with anytime inference, 2022. URL <https://arxiv.org/abs/2110.10031>. 5
- [15] Matthias De Lange, Gido van de Ven, and Tinne Tuytelaars. Continual evaluation for lifelong learning: Identifying the stability gap, 2023. URL <https://arxiv.org/abs/2205.13452>. 1, 2, 4, 6, 10, 11
- [16] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning, 2022. URL <https://arxiv.org/abs/1706.08840>. 2, 4, 6
- [17] Michael McCloskey and Neil J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24:104–169, 1989. 1
- [18] Martial Mermillod, Aurélia Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in psychology*, 4:504, 08 2013. doi: 10.3389/fpsyg.2013.00504. 1
- [19] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Dilan Gorur, Razvan Pascanu, and Hassan Ghasemzadeh. Linear mode connectivity in multitask and continual learning, 2020. URL <https://arxiv.org/abs/2010.04495>. 5
- [20] Roger Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97:285–308, 04 1990. doi: 10.1037/0033-295X.97.2.285. 1
- [21] Albin Soutif-Cormerais, Antonio Carta, and Joost Van de Weijer. Improving online continual learning performance and stability with temporal ensembles, 2023. URL <https://arxiv.org/abs/2306.16817>. 5
- [22] Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. *Continual learning and catastrophic forgetting*, page 153–168. Elsevier, 2025. ISBN 9780443157554. doi: 10.1016/b978-0-443-15754-7.00073-0. URL <http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0>. 1, 3
- [23] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Trans. Math. Softw.*, 11(1): 37–57, March 1985. ISSN 0098-3500. doi: 10.1145/3147.3165. URL <https://doi.org/10.1145/3147.3165>. 4
- [24] Wojciech Łapacz, Daniel Marczak, Filip Szatkowski, and Tomasz Trzcíński. Exploring the stability gap in continual learning: The role of the classification head, 2024. URL <https://arxiv.org/abs/2411.04723>. 5

Detailed Step-by-Step Proof of the Trajectory Bound

The Mathematical Setup & Definitions

1. The Combined Loss. The total loss function at any point $\lambda \in [0, 1]$ in the curriculum is defined as:

$$L_\lambda(\theta) = L_{\text{replay}}(\theta) + \lambda \cdot L_{\text{current}}(\theta) \quad (.4)$$

2. Defining the Hessians (∇^2). When we take the derivative of a gradient (∇), we obtain the second derivative matrix, called the Hessian (H).

- $H_{\text{replay}} = \nabla^2 L_{\text{replay}}(\theta)$ (The curvature of the old task)
- $H_{\text{current}} = \nabla^2 L_{\text{current}}(\theta)$ (The curvature of the new task)

Therefore, the Hessian of our combined loss, H_λ , is defined by applying the second derivative operator to the total loss function:

$$\begin{aligned} H_\lambda &= \nabla^2 [L_{\text{replay}}(\theta) + \lambda \cdot L_{\text{current}}(\theta)] \\ H_\lambda &= \nabla^2 L_{\text{replay}}(\theta) + \lambda \cdot \nabla^2 L_{\text{current}}(\theta) \\ H_\lambda &= H_{\text{replay}} + \lambda \cdot H_{\text{current}} \end{aligned}$$

Part A: The Perfect Path Never Spikes

Let $\Theta^*(\lambda)$ be the ‘‘Optimum Path’’—the exact parameter state that perfectly minimises $L_\lambda(\theta)$ for any given λ .

Step 1: The First-Order Condition. Because $\Theta^*(\lambda)$ is the perfect minimum, the gradient (slope) of the total loss evaluated at that exact point must be zero. We define this as our starting equilibrium:

$$\begin{aligned} \nabla [L_\lambda(\Theta^*(\lambda))] &= 0 \\ \nabla [L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot L_{\text{current}}(\Theta^*(\lambda))] &= 0 \end{aligned}$$

Distributing the gradient operator to both terms gives us the fundamental First-Order Condition:

$$\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0 \quad (.5)$$

Step 2: Implicit Differentiation to Find Path Movement. We want to know how the optimal path Θ^* moves as λ changes. We apply the derivative operator $\frac{d}{d\lambda}$ to the entire First-Order Condition:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = \frac{d}{d\lambda} [0]$$

Distributing the operator to each term:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda))] + \frac{d}{d\lambda} [\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = 0$$

Evaluating the first term via the multivariable chain rule:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda))] = \nabla^2 L_{\text{replay}}(\Theta^*(\lambda)) \cdot \frac{d\Theta^*}{d\lambda}$$

Evaluating the second term via the product rule and chain rule:

$$\frac{d}{d\lambda} [\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = (1) \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) + \lambda \cdot \left(\nabla^2 L_{\text{current}}(\Theta^*(\lambda)) \cdot \frac{d\Theta^*}{d\lambda} \right)$$

Substituting these expanded terms back into the equation:

$$\left[\nabla^2 L_{\text{replay}}(\Theta^*) \cdot \frac{d\Theta^*}{d\lambda} \right] + \left[\nabla L_{\text{current}}(\Theta^*) + \lambda \cdot \nabla^2 L_{\text{current}}(\Theta^*) \cdot \frac{d\Theta^*}{d\lambda} \right] = 0$$

Grouping the terms that share the path movement $\frac{d\Theta^*}{d\lambda}$:

$$(\nabla^2 L_{\text{replay}}(\Theta^*) + \lambda \cdot \nabla^2 L_{\text{current}}(\Theta^*)) \cdot \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}}(\Theta^*) = 0$$

Recognising the definition of the combined Hessian H_λ inside the parentheses:

$$H_\lambda \cdot \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0$$

Solving algebraically for the path movement $\frac{d\Theta^*}{d\lambda}$:

$$\frac{d\Theta^*}{d\lambda} = -H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) \quad (.6)$$

Step 3: Calculating the Change in Task 0 Loss. How does the Replay loss specifically change as λ increases along this perfect path? We apply the derivative operator:

$$\frac{d}{d\lambda} [L_{\text{replay}}(\Theta^*(\lambda))] = \nabla L_{\text{replay}}(\Theta^*(\lambda))^\top \cdot \frac{d\Theta^*(\lambda)}{d\lambda}$$

We perform two substitutions here. First, from (.5), we know $\nabla L_{\text{replay}} = -\lambda \cdot \nabla L_{\text{current}}$. Second, from (.6), we substitute the path movement.

$$\begin{aligned} \frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) &= (-\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)))^\top \cdot (-H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))) \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ &= \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))^\top \cdot H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \end{aligned}$$

Because H_λ is positive-definite (strict minimum assumption), H_λ^{-1} is positive-definite. The quadratic form $v^\top M v$ is therefore guaranteed to be ≥ 0 . Since $\lambda \geq 0$, the entire equation satisfies:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0$$

Conclusion for Part A: The Replay loss mathematically never decreases along the perfect path, meaning it can never overshoot its final joint-optimum value.

Part B: The Clumsy Network Tracks the Perfect Path

Let $\theta(t)$ be the actual position of the model parameters at time t . We define the tracking error $\delta(t)$:

$$\delta(t) = \theta(t) - \Theta^*(\lambda(t))$$

Step 1: Differentiating the Error. We apply the time derivative operator $\frac{d}{dt}$ to our error definition:

$$\frac{d}{dt} [\delta(t)] = \frac{d}{dt} [\theta(t)] - \frac{d}{dt} [\Theta^*(\lambda(t))]$$

The model moves via gradient descent, so $\frac{d\theta}{dt} = -\nabla L_{\lambda(t)}(\theta(t))$. The perfect path moves via the chain rule $\frac{d\Theta^*}{dt} = \frac{d\Theta^*}{d\lambda} \cdot \frac{d\lambda}{dt}$. Because the curriculum has a constant velocity over N steps, $\frac{d\lambda}{dt} = \frac{1}{N}$.

$$\frac{d\delta}{dt} = -\nabla L_{\lambda(t)}(\theta(t)) - \left(\frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \right)$$

Step 2: Taylor Expansion of the Actual Gradient. We approximate the gradient at the actual position $\theta(t)$ by Taylor-expanding it around the perfect position Θ^* :

$$\nabla L_\lambda(\theta) \approx \nabla L_\lambda(\Theta^*) + \nabla [\nabla L_\lambda(\Theta^*)] \cdot (\theta - \Theta^*)$$

Since the gradient at the perfect minimum is zero ($\nabla L_\lambda(\Theta^*) = 0$) and the derivative of the gradient is the Hessian ($\nabla^2 L_\lambda = H_\lambda$), this collapses to:

$$\nabla L_\lambda(\theta) \approx H_\lambda \cdot \delta$$

Step 3: The Final Steady State. Substitute this expanded gradient back into our differential equation:

At a steady-state tracking lag, the error stops growing ($\frac{d\delta}{dt} \approx 0$):

$$\begin{aligned} 0 &\approx -H_\lambda \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ H_\lambda \cdot \delta &\approx -\frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ \delta &\approx -H_\lambda^{-1} \cdot \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \end{aligned}$$

Taking the norm, we see the lag distance is mathematically bounded by the curriculum length N :

$$\|\delta\| = O\left(\frac{1}{N}\right) \quad (.7)$$

The Grand Finale: Expanding the Actual Loss

To find the maximum Replay loss the model will actually experience, we Taylor expand the loss of the actual trajectory around the perfect position:

$$\begin{aligned} L_{\text{replay}}(\theta) &\approx L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top \cdot (\theta - \Theta^*) \\ L_{\text{replay}}(\theta) &\approx L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top \cdot \delta \end{aligned}$$

From Part A, $L_{\text{replay}}(\Theta^*)$ cannot exceed the final joint optimum loss $L_{\text{replay}}(\theta_{\text{joint}}^*)$. From Part B, the lag δ is $O(1/N)$. Substituting these limits yields the final mathematical guarantee:

$$\sup_t L_{\text{replay}}(\theta(t)) \leq L_{\text{replay}}(\theta_{\text{joint}}^*) + O\left(\frac{1}{N}\right) \quad (.8)$$

Conclusion: The transient stability gap is entirely contained within the $O(1/N)$ term, meaning a sufficiently slow continuous curriculum eliminates the discrete-time overshoot.